

**SIMATS ENGINEERING
ASSESSMENT TOOL 3**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: *Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: 2

Duration : 45 Minutes

Total Marks: 20

Annexure A

Descriptive Written Test

Code of Conduct:

*I **BALAJI D** (Name) / **192512368** (Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*Department: **B E ECE***

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **BALAJI D**

Annexure A – Descriptive Written Test

Q1.

A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

Answer

1. Simple Reflex Agent

A simple reflex agent selects actions based only on the current percept, using fixed condition–action (if–then) rules, with no memory of the past and no model of the world. Within the smart home, a simple reflex agent could handle very direct, immediate responses – for example, “IF motion is detected in a dark room THEN turn on the light”, or “IF the door sensor is triggered while the security mode is armed THEN sound the alarm”. This works well for instantaneous, unambiguous triggers, but the agent cannot reason about context: it would turn the lights on even if it is already bright daylight outside, and it cannot adapt its behaviour based on who is home or what time it is, because it has no internal memory of the environment.

2. Model-Based Reflex Agent

A model-based agent maintains an internal model of the world that is updated as new percepts arrive, allowing it to keep track of aspects of the environment that are not currently visible. In the smart home, this agent would track state such as “which rooms are currently occupied”, “what the temperature trend has been over the last hour”, or “whether it is day or night”, even between sensor readings. This allows smarter decisions than a simple reflex agent – for instance, the system can avoid switching security mode off just because a motion sensor briefly failed to trigger, since it remembers that the house was recently marked occupied. It still, however, does not explicitly reason about future goals; it only maintains and reacts to an internal state.

3. Goal-Based Agent

A goal-based agent chooses actions by considering which sequence of actions will achieve an explicit goal, rather than reacting only to the current state. In the smart home, a goal such as “bring the living room to 22°C by 6 PM when the family usually arrives” requires the agent to plan ahead – deciding when to start the heating or cooling system based on how long it typically takes to reach the target temperature. Similarly, a security goal such as “ensure the house is fully secured before the last occupant leaves” requires the agent to check multiple conditions (all doors locked, windows closed, alarm armed) rather than responding to a single percept. This gives the system foresight and purposeful behaviour, but by itself a goal-based agent only tells us whether a goal is achieved or not – it does not naturally compare between several acceptable ways of reaching the goal.

4. Utility-Based Agent

A utility-based agent goes a step further than a goal-based agent by assigning a utility (a numeric measure of “desirability”) to different possible states or outcomes, allowing it to choose the best option when several actions could all satisfy a goal, or when goals conflict. In the smart home, comfort, energy efficiency and security often pull in different directions – for example, keeping the air-conditioning running constantly maximises comfort but wastes energy, while turning it off entirely saves energy but sacrifices comfort. A utility-based agent can weigh these trade-offs (e.g., $utility = w_1 \times comfort_score + w_2 \times energy_saving_score + w_3 \times security_score$) and choose the action that maximises overall utility, such as pre-cooling the house only slightly before occupants are expected, rather than maintaining a constant temperature all day.

Recommended Approach: A Hybrid Model-Based, Goal-Based and Utility-Based Agent

No single agent type on its own is sufficient for this system. A purely reflex-based design cannot learn preferences or handle competing objectives; a purely goal-based design cannot rank multiple acceptable solutions; and a system with no internal model cannot make sensible decisions when sensor data is temporarily missing or when reasoning about trends over time (such as learning that a user prefers a warmer house in the evening).

The most effective design is therefore a hybrid, layered architecture: a model-based core continuously maintains an internal representation of the home (occupancy, temperature history, time of day, learned user routines); on top of this, a goal layer defines explicit objectives (maintain comfortable temperature when occupied, ensure security when unoccupied, minimise energy waste); and a utility layer resolves conflicts between these goals and ranks candidate actions by combining comfort, energy and security scores, using the learned user-preference data to personalise the weighting over time. Fast, safety-critical responses (e.g., an intrusion alarm or a smoke/fire trigger) can still be implemented as simple reflex rules that override the utility-based planning layer, so the system remains instantly responsive for critical events while being intelligent and adaptive for everyday comfort and efficiency decisions. This layered hybrid approach gives the smart home system the immediacy of reflex behaviour, the situational awareness of a model-based agent, the purposefulness of a goal-based agent, and the nuanced trade-off handling of a utility-based agent – making it the most effective overall design.

Q2.

A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Answer

Applying Uniform Cost Search (UCS) to the Maze

Uniform Cost Search explores the maze by always expanding the unvisited node with the lowest cumulative path cost $g(n)$ so far, using a priority queue (min-heap) ordered by $g(n)$. For the robot in the maze, each move (e.g., one step in a direction) can be given a cost of 1, or a higher cost if certain cells are harder to traverse (e.g., rough terrain). Because UCS always expands the cheapest frontier node first, it guarantees that when the exit cell is finally expanded, the path found to it is the lowest-cost path – which is exactly what is needed if the maze has cells with varying movement costs, not just a simple step count.

Applying Iterative Deepening Search (IDS) to the Maze

Iterative Deepening Search repeatedly performs a depth-limited depth-first search, increasing the depth limit by one each iteration (limit = 0, 1, 2, ...) until the exit is found. For the robot, this means it first checks whether the exit is reachable in 0 moves, then in at most 1 move, then at most 2 moves, and so on, re-exploring from the start each time with the new limit. This is particularly useful when the maze's depth (the length of the shortest path) is unknown in advance, since IDS does not require deciding a search depth ahead of time, and it will find the shallowest (fewest-move) solution first, similar to breadth-first search.

Advantages and Disadvantages (Time and Space Complexity)

Algorithm	Time Complexity	Space Complexity	Advantages	Disadvantages
UCS	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$	Guarantees the least-cost path even with varying step costs; complete if costs are positive.	Can explore a very large frontier (high memory use); may be slow if many low-cost paths exist before reaching the goal (C is large).
IDS	$O(b^d)$	$O(b \times d)$	Combines the low memory use of DFS with the completeness/optimality (for unit-cost problems) of BFS; does not need to know the maze depth in advance.	Repeatedly re-explores shallow parts of the search space in each iteration, causing redundant work when all step costs are equal.

In summary, UCS is preferable when different moves in the maze have different costs and the optimal (cheapest) path matters, at the expense of higher memory usage. IDS is preferable when memory is limited and all moves cost the same, since its space complexity grows only linearly with search depth, though it repeats work across iterations.

Relating the Problem to Intelligent Agent Types

The maze-solving robot cannot be a simple reflex agent, because reacting only to the current percept (e.g., “wall on the left, so turn right”) can easily lead to infinite loops or failure to find the exit in a complex maze – the robot has no memory of where it has already been. A model-based reflex agent is a minimum requirement: the robot must maintain an internal map of visited cells and walls discovered so far, so it can avoid revisiting the same dead ends. However, to actually solve the maze in a systematic and complete way, the robot needs to be a goal-based agent – its goal is explicitly “reach the exit cell”, and it uses a search algorithm (such as UCS or IDS) over its internal map/model to plan a sequence of actions that achieves this goal, rather than simply reacting move by move. If the maze has cells with different traversal costs (e.g., some areas are slower or riskier to move through) and the robot must find not just any path but the best one, then it additionally becomes a utility-based agent, since it is now comparing paths by a cost/utility measure rather than only checking whether the goal is reached.

Agent design directly affects decision-making here: a purely reactive design risks looping or failing entirely in a maze with cycles, a model-based design enables systematic exploration without repeating dead ends, and a goal/utility-based design determines whether the robot merely finds a path or finds the best path – which matters if, for example, battery life or time is limited and shorter/cheaper paths are strongly preferred.

Recommended Combination and Justification

The most effective combination is a model-based, goal-based (extended with utility where step costs differ) agent using Uniform Cost Search as its underlying search strategy. The model-based component is essential so the robot remembers which cells have already been explored, preventing infinite loops and redundant work in a maze that may contain cycles. The goal-based component gives the robot the explicit objective of reaching the exit and lets it use systematic search rather than blind reaction. UCS is the preferred search strategy over IDS whenever the maze cells can have unequal traversal costs (e.g., rough vs. smooth terrain, or risk of obstacles), because UCS is guaranteed to find the lowest-cost path, which directly maximises the robot's efficiency and safety – exactly the kind of trade-off a utility-based agent needs to reason about. If, on the other hand, the maze is known to have uniform step costs and the robot has strict memory constraints (e.g., a very small embedded controller), IDS would be the more practical choice, since its linear space complexity $O(b \times d)$ is far more memory-efficient than UCS's exponential frontier, at the cost of some repeated computation. Given that real mazes and physical robots typically do have varying terrain costs and modern robot controllers have adequate memory, the model-based, goal/utility-based agent combined with Uniform Cost Search is the most effective and well-justified overall design for this problem.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	5
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	5
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	4
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	3
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	2
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	1

Total Marks: 20